

2024-2025 学年下学期面向对象程序设计 (C++)

选择题部分参考文档

一、面向对象程序设计概述

1. 下面说法中，错误的是：(D)
 - A. 抽象是通过特定的实例（对象）抽取共同性质后形成概念的过程
 - B. 封装是指把数据和实现操作的代码集中起来放在对象内部，并尽可能隐蔽对象的内部细节
 - C. 在面向对象程序设计语言中，某一类可以继承另外一类的特征和能力
 - D. 多继承是指一个基类派生出多个派生类的继承关系
2. 下面选项中，哪一项不是面向对象程序设计的基本特征：(C)
 - A. 继承
 - B. 封装
 - C. 归纳
 - D. 多态
3. 下面关于消息的说法中，错误的是：(D)
 - A. 在面向对象程序设计中，对象之间的联系称为对象的交互。消息机制允许一个对象与另一个对象进行交互。
 - B. 同一个对象可以接收不同形式的多个消息，做出不同的响应
 - C. 相同形式的消息可以传递给不同的对象，所做出的响应可以是不同的
 - D. 对象对消息的响应是必需的
4. 下列选项中，哪个不是面向对象系统的特性？(C)
 - A. 可重用性好
 - B. 可扩充性好
 - C. 模块化
 - D. 继承性
5. 下述面向对象抽象的原理中，哪个是不对的。(D)
 - A. 数据抽象
 - B. 行为共享
 - C. 继承
 - D. 兼容
6. 关于类定义格式的描述中，哪个是错的。(D)
 - A. 一般类的定义格式分为说明部分和实现部分
 - B. 一般类中包含有数据成员和成员函数
 - C. 类中成员有三种访问数据：公有、私有和保护
 - D. 成员函数都应是公有的、数据成员都应是私有的
7. 面向对象程序设计思想的主要特征中不包括 (D)
 - A. 封装性
 - B. 多态性
 - C. 继承性
 - D. 功能分解，逐步求精

二、C++ 概述

1. 下列符号中，正确的 C++ 标识符是 (D)

- A. enum B. 2c C. foo-9 D. _32

2. 关于 C++ 和 C 语言的描述中，(C) 是错误的？

- A. C 是 C++ 的一个子集 B. C 程序在 C++ 环境可以运行
C. C++ 程序在 C 环境可以运行 D. C++ 是面向对象的而 C 是面向过程的

3. 关于 new 运算符的下列描述中，(D) 是错误的

- A. 它可以用来动态创建对象和对象数组
B. 使用它创建的对象或对象数组可以使用运算符 delete 删除
C. 使用它创建对象时要调用构造函数
D. 使用它创建对象数组时必须指定初始值

4. 有如下函数定义：

```
void func (int a,int&b) {a++; b++;}
```

若执行代码段：

```
int x=0 ,y=1;  
func (x,y) ;
```

则变量 x 和 y 值分别是：(C)

- A. 0 和 1 B. 1 和 1 C. 0 和 2 D. 1 和 2

5. C++ 是 (C)

- A. 面向对象的程序设计语言
B. 面向过程的程序设计语言
C. 既支持面向对象的程序设计又支持面向过程的程序设计的混合型语言
D. 非结构化的程序设计语言

6. 下列描述中，(C) 是错误的。

- A. 内联函数主要解决程序的运行效率问题
B. 内联函数的定义必须现在内联函数第一次被调用之前
C. 内联函数中可以包括各种语句
D. 对内联函数不可以进行异常接口声明

7. 若有下面的函数调用:

```
fun(a+b, 3, max(n-1, b))
```

则 fun 的实参个数是: (A)

- A. 3 B. 4 C. 5 D. 6

8. 下面关于 C++ 的特点中, 错误的是: (D)

- A. C++ 是 C 的超集, 许多 C 代码不经修改就可以为 C++ 所用
B. C++ 对 C 的功能作了不少扩充, 缩写出来的程序比 C 更安全, 可读性更好, 代码结构更合理
C. C++ 几乎支持所有的面向对象程序设计的特征
D. C++ 是一个纯正的面向对象的语言

9. 下列选项中, 哪一项不是面向对象程序设计的优点: (C)

- A. 可提高程序的重用性, 可改善程序的可维护性
B. 可控制程序的复杂性, 能够更好地支持大型程序设计
C. 可增强程序的健壮性, 增加程序设计的复杂度
D. 增强了计算机处理信息的范围, 能很好地适应新的硬件环境

10. 下面的类型声明中正确的是 (D)

- A. `int &a[4];` B. `int *&p;` C. `int &&q;` D. `int i, *p=&i;`

11. 有如下程序段

```
int i =0, j=1;  
int&r=i ; // ①  
r =j; //②  
int*p=&i ; //③  
*p=&r ; //④
```

其中会产生编译错误的语句是: (A)

- A. ④ B. ③ C. ② D. ①

12. 下面程序的运行结果是: (C)

```
#include<iostream.h>  
int &func(int &num)  
{ num++;  
return num; }
```

```
void main()
{   int n1,n2=5;
    n1=func(n2);
    cout<<n1<<" "<<n2<<endl; }
```

- A. 5 6 B. 6 5 C. 6 6 D. 5 5

13. 在（A）情况下适宜采用内联函数

- A. 函数代码小，频繁调用 B. 函数代码多，频繁调用
C. 函数体含有递归语句 D. 函数体含有循环语句

14. 有如下程序：

```
#include<iostream>
using namespace std;
int main()
{
    int *p;*p = 9;
    cout<<"The value at p:"<<*p;
    return 0;}
```

编译运行程序将出现的情况是：（D）

- A. 编译时出现语法错误，不能生成可执行文件
B. 运行时一定输出：The value at p: 9
C. 运行时一定输出：The value at p: *9
D. 运行时有可能出错

15. 有 `int *p=new int[10];` 的语句，则释放 p 所指向的空间的语句为：（B）

- A. `delete *p;` B. `delete p;` C. `del *p;` D. `del p;`

16. 下列选项中哪一项不是程序设计范型所涉及的？（D）

- A. 程序的规范 B. 程序的模型 C. 程序的风格 D. 程序语言的名称

17. 下列表示引用的方法中，哪个是正确的？已知：`int a=1000;`（A）

- A. `int &x=a;` B. `char &y;` C. `int &z=1000;` D. `float &t=&a;`

18. 下列带缺省值参数的函数说明中，正确的说明是：（B）

- A. `int Fun(int x=1,int y=2,int z);` B. `int Fun(int x,int y=2,int z=3);`
C. `int Fun(int x,int y=2,int z);` D. `int Fun(int x=1,int y,int z=3);`

19. 在 C++ 中, 一个函数为 `void f(int=1,char='a')`, 另一个函数为 `void f(int)`, 则它们: (B)
- A. 不能在同一程序中定义
 - B. 可以在同一程序中定义并可重载, 但运行 `f(5)` 时会出现语法错误
 - C. 可以在同一程序中定义, 但不可重载
 - D. 其他说法都不正确
20. 在 C++ 类与对象的应用中, 运算符 `new` 的作用是: (C)
- A. 创建名为 `new` 的对象
 - B. 获取一个新类的内存
 - C. 返回指向所创建对象的指针, 并为创建的对象分配内存空间
 - D. 返回为所创建的对象分配内存的大小
21. 在 C++ 中, 关于下列设置参数默认值的描述中, 哪个是正确的。 (C)
- A. 不允许设置参数的默认值
 - B. 设置参数默认值只能在定义函数时设置
 - C. 设置参数默认值时, 应该是先设置右边的, 再设置左边的
 - D. 设置参数默认值时, 应该全部参数都设置
22. 下面标识符中正确的是 (A)。
- A. `_abc`
 - B. `3ab`
 - C. `int`
 - D. `+ab`
23. 下列关于 C++ 函数的叙述中, 正确的是: (C)
- A. 每个函数至少要具有一个参数
 - B. 每个函数都必须返回一个值
 - C. 函数 (除 `main` 函数外) 在被调用之前必须先声明
 - D. 函数不能自己调用自己
24. 关于 `delete` 运算符的下列描述中, (C) 是错误的
- A. 它必须用于 `new` 返回的指针
 - B. 使用它删除对象时要调用析构函数
 - C. 对一个指针可以使用多次该运算符
 - D. 指针名前只有一对方括号符号, 不管所删除数组的维数
25. 对于语句 `cout<<endl<<x` 中的各个组成部分, 下列叙述中错误的是: (D)
- A. “`cout`” 是一个输出流对象
 - B. “`endl`” 的作用是输出回车换行
 - C. “`x`” 是一个变量
 - D. “`<<`” 称作提取运算符

26. 下列语句中错误的是：(D)

- A. `int *p=new int(10);`
- B. `int *p=new int[10];`
- C. `int *p=new int;`
- D. `int *p=new int[40](0);`

27. “类名:: 成员名” 中作用域运算符的功能是：(D)

- A. 标识类的作用范围
- B. 标识成员是全局的
- C. 给定作用域的大小的
- D. 标识成员是属于哪个类的

28. 下列选项中错误的是：(B)

- A. C 语言的类型检查机制相对较弱，这使得程序中的一些错误不能在编译阶段由编译器检查出来
- B. C 语言本身有很多支持代码重用的语言结构
- C. C 语言是面向过程的编程语言
- D. C++ 弥补了 C 语言不支持代码重用、不适宜开发大型软件的不足

29. 下列语句中，错误的是 (D)

- A. `const int buffer=256;`
- B. `const double *point;`
- C. `int const buffer=256;`
- D. `double * const point;`

30. 在一个函数中，要求通过函数来实现一种不太复杂的功能，并且要求加快执行速度，选用：(A)

- A. 内联函数
- B. 重载函数
- C. 递归调用
- D. 嵌套调用

31. 有如下程序：

```
#include<iostream>
using namespace std;
int main()
{   void function(double val);
    double val;
    function(val);
    cout<<val;
    return 0;}
void function(double val)
{   val = 3; }
```

编译运行这个程序将出现的情况是：(D)

- A. 编译出错，无法运行
- B. 输出：3
- C. 输出：3.0
- D. 输出一个不确定的数

32. 若定义: `string str;` 语句 `cin>>str;` 执行时, 从键盘输入: Microsoft Visual Studio 6.0!, `str=` (B)

A. Microsoft Visual Studio 6.0!

B. Microsoft

C. Microsoft Visual

D. Microsoft Visual Studio 6.0

33. 关于函数重载, 下列叙述中错误的是: (C)

A. 重载函数的函数名必须相同

B. 重载函数必须在参数个数、参数类型或参数顺序上有所不同

C. 重载函数的返回值类型必须相同

D. 重载函数的函数体可以有所不同

三、类和对象

1. 有如下类定义

```
#include<iostream.h>
class point
{   int x,y;
    public:
    point():x(0),y(0){}
    point(int x1,int y1=0):x(x1),y(y1){}};
```

若执行语句: `point a(1),b[2],*c;`, 则 `point` 类的构造函数被调用的次数是: (C)

- A. 1 B. 2 C. 3 D. 4

2. 类中有一些函数的作用是支持其他函数的操作, 是类中其他成员的工具函数, 且不能被类外访问; 这些函数的成员访问限定符是: (C)

- A. `public` B. `protected` C. `private` D. 其他都不是

3. 在类定义的外部, 可以被访问的成员有: (C)

- A. 所有类成员 B. `private` 的类成员
C. `public` 的类成员 D. `public` 或 `private` 的类成员

4. 设有以下类的定义:

```
class Ex
{   int x;
    public: void setx(int t=0);
};
```

若在类外定义成员函数 `setx()`, 以下定义形式中正确的是: (B)

- A. `void setx(int t) ...` B. `void Ex::setx(int t) ...`
C. `Ex::void setx(int t) ...` D. `void Ex::setx() ...`

5. 有如下类定义:

```
class Test
{   public:
    Test(){ a = 0; c = 0;} //①
    int f(int a) const {this->a = a;} //②
    static int g(){return a;} //③
    void h(int b){Test::b = b;} //④
    private:
```



```

int a;
static int b;
const int c;};
    int Test::b = 0;

```

在标注号码的行中，能被正确编译的是 (D)

- A. ① B. ② C. ③ D. ④

6. 在下列函数原型中，可以作为类 AA 构造函数的是：(D)

- A. void AA(int); B. int AA(); C. AA(int) const; D. AA(int);

7. 假设已经有定义 `char *const name="chen";`，下面语句中正确的是：(A)

- A. `name[3]='q';` B. `name="lin";`
 C. `name=new char[5];` D. `name=new char('q');`

8. 下列的各类函数中，哪个不是类的成员函数。(C)

- A. 构造函数 B. 析构函数
 C. 友元函数 D. 拷贝初始化构造函数

9. 由于常对象不能被更新，因此 (A)

- A. 通过常对象只能调用它的常成员函数
 B. 通过常对象只能调用静态成员函数
 C. 常对象的成员都是常成员
 D. 通过常对象可以调用任何不改变对象值的成员函数

10. 假设已经有定义 `const char *const name="chen";`，下面的语句正确的是：(D)

- A. `name[3]='a';` B. `name="lin";`
 C. `name=new char[5];` D. `cout<<name[3];`

11. 下列说法中，错误的是：(B)

- A. 不论成员函数在类内定义还是在类外定义，成员函数的代码段的存储方式都是相同的，都不占对象的存储空间
 B. inline 函数的代码段会占用对象的存储空间
 C. inline 函数只影响程序的执行效率，而与成员函数是否占用对象的存储空间无关
 D. 不论是否用 inline 声明，成员函数的代码段都不占用对象的存储空间

12. 在下面有关友元函数的描述中，正确的说法是：(A)

- A. 友元函数是独立于当前类的外部函数

- B. 一个友元函数不能同时定义为两个类的友元函数
- C. 友元函数必须在类的外部定义
- D. 在外部定义友元函数时，必须加关键字 friend
13. 在下列哪种情况下不会调用拷贝构造函数：(D)
- A. 用一个对象去初始化本类的另一个对象时
- B. 函数的形参是类的对象，在进行形参和实参的结合时
- C. 函数的返回值是类的对象，函数执行完返回时
- D. 将类的一个对象赋值给另一个本类的对象时
14. 类 A 是类 B 的友元，类 B 是类 C 的友元，则：(D)
- A. 类 B 是类 A 的友元
- B. 类 C 是类 A 的友元
- C. 类 A 是类 C 的友元
- D. 其他都不对
15. 已知 Time 类有一公有函数，声明为：void set_time();, 定义指向此函数的指针变量的语句为：(D)
- A. void (*p)()
- B. void (Time *p)()
- C. void Time::*p()
- D. void (Time::*p)()
16. 关于友元函数的描述中，错误的是：(B)
- A. 友元函数不是成员函数
- B. 友元函数只能访问类中私有成员
- C. 友元函数破坏隐藏性，尽量少用
- D. 友元函数说明在类体内，使用关键字 friend
17.

```
#include <iostream>
using namespace std;
class XA{
    int a;
public:
    static int b;
    XA(int aa):a(aa) {b++;}
    ~XA(){}
    int get(){return a;}
};
int XA::b=0;
int main(){
    XA d1(2),d2(3);
    cout<<d1.get()+d2.get()+XA::b<<endl;
    return 0;
```

}

运行时的输出结果是 (C)

- A. 5 B. 6 C. 7 D. 8

18. 在下面有关析构函数特征的描述中，正确的是：(C)

- A. 一个类中可以定义多个析构函数 B. 析构函数名与类名完全相同
C. 析构函数不能指定返回类型 D. 析构函数可以有一个或多个参数

19. 下列说法中，正确的是：(B)

- A. 一个对象所占的空间大小取决于该对象中数据成员与成员函数所占的空间有关
B. 一个对象所占的空间大小取决于该对象中数据成员所占的空间
C. 一个对象所占的空间大小取决于该对象中成员函数所占的空间
D. 一个对象所占的空间大小与该对象中数据成员所占空间无关

20. 下列说明中 `const char *ptr;`, `ptr` 应该是：(A)

- A. 指向字符常量的指针 B. 指向字符的常量指针
C. 指向字符串常量的指针 D. 指向字符串的常量指针

21. 拷贝构造函数具有的下列特点中，(D) 是错误的

- A. 如果一个类中没有定义拷贝构造函数时，系统将自动生成一个默认的
B. 拷贝构造函数只有一个参数，并且是该类对象的引用
C. 拷贝构造函数是一种成员函数
D. 拷贝构造函数的名字不能用类名

22. 有如下头文件：

```
int f1();  
static int f2();  
class MA{  
public:  
    int f3();  
    static int f4();  
};
```

在所描述的函数中，具有隐含的 `this` 指针的是：(C)

- A. `f1` B. `f2` C. `f3` D. `f4`

23. 通常拷贝构造函数的参数是：(C)

- A. 某个对象名 B. 某个对象的成员名
C. 某个对象的引用名 D. 某个对象的指针名

24. 下列选项中，哪一项不是现实世界中对象的特性：(B)

- A. 每一个对象必须有一个名字以区别于其他对象
- B. 用函数来描述对象的某些特征
- C. 有一组操作，每组操作决定对象的一种行为
- D. 对象的行为可以分为两类：一类是用于自身的行为，另一类是作用于其他对象的行为。

25. 运行下列程序后，"constructing A!" 和 "destructing A!" 分别输出几次？(B)

```
class A
{ int x;
public:
  A(){cout<<" constructing A!"<<endl;}
  ~A(){cout<<"destructing A! " <<endl;}
};

void main()
{   A a[2];
    A *p=new A;
    delete p;
}
```

- A. 2 次, 2 次 B. 3 次, 3 次 C. 1 次, 3 次 D. 3 次, 1 次

26. 关于友元类的描述中，错误的是：(C)

- A. 友元类中的成员函数都是友元函数
- B. 友元类被说明在一个类中，它与访问权限无关
- C. 友元类是被定义在某个类中的嵌套类
- D. 如果 Y 是类 X 的友元，类 X 不一定是类 Y 的友元

27. 有如下类定义：

```
class Foo
{
public:
  Foo(int v):value(v){} //①
  ~Foo(){} //②
  Foo(){} //③
private:
  int value = 0; //④
};
```

其中存在语法错误的行是 (C++ 98 标准): (D)

- A. ① B. ② C. ③ D. ④

28. 有如下类定义:

```
#include<iostream>
using namespace std;
class A{
public:
    static int a;
    void init(){a=1;}
    A(int a=2) {init();a++;};
int A::a=0;
A obj;
int main(){
    cout<<obj.a;
    return 0;}
```

运行时输出的结果是: (B)

- A. 0 B. 1 C. 2 D. 3

29. 关于成员函数特征的下列描述中, 哪个是正确的。(C)

- A. 成员函数一定是内联函数
- B. 所有的成员函数都以重载
- C. 成员函数可以设置缺省参数值
- D. 数据成员可以是静态的, 成员函数不可以是静态的

30. 下列程序的运行结果为: (C)

```
#include<iostream.h>
int i=0;
class A
{ public: A(){i++;} };
void main()
{ A a,b[3],*c;
  c=b;
  cout<<i<<endl;}
```

- A. 2 B. 3 C. 4 D. 5

31. 在下列函数原型中，可以作为类 AA 构造函数的是：(D)

A. void AA(int); B. int AA(); C. AA(int) const; D. AA(int);

32. 关于 new 运算符的下列描述中，(D) 是错的。

- A. 它可以用来动态创建对象和对象数组
- B. 使用它创建的对象或对象数组可以使用运算符 delete 删除
- C. 使用它创建对象时要调用构造函数
- D. 使用它创建对象数组时必须指定初始值

33. 下面是对类 MyClass 的定义，对定义中语句描述正确的是：(B)

```
class MyClass
{public:
    void MyClass(int a) {X=a;} //①
    int f(int a,int b) //②
    {X=a;
      Y=b;}
    int f(int a,int b,int c=0) //③
    {X=a; Y=b; Z=c;}
    static void g() {X=10;} //④
private:
    int X,Y,Z;};
```

- A. 语句①是类 MyClass 的构造函数定义 B. 语句②和语句③实现类成员函数的重载
- C. 语句④实现对类成员变量 X 的更新操作 D. 语句①、②、③和④都不正确

四、派生类与继承

1. 在公有继承的情况下，允许派生类直接访问的基类成员包括：(B)

- A. 公有成员
- B. 公有成员和保护成员
- C. 公有成员、保护成员和私有成员
- D. 保护成员

2. 派生类的成员函数不能访问基类的：(C)

- A. 公有成员和保护成员
- B. 公有成员
- C. 私有成员
- D. 保护成员

3. 分析下面的 C++ 代码段：

```
class Employee
{ private: int a;
  protected: int b;
  public: int c; };
class Director:public Employee{};
```

在 main() 中，下列 (C) 操作是正确的。

- A. Employee obj; obj.b=1;
- B. Director boj; obj.b=10;
- C. Employee obj; obj.c=3;
- D. Director obj; obj.a=20;

4. 多继承派生类建立对象时，(B) 被最先调用。

- A. 派生类自己的构造函数
- B. 虚基类的构造函数
- C. 非虚基类的构造函数
- D. 派生类中子对象类的构造函数

5.

```
#include <iostream>
using namespace std;
class Base{
public:
void fun() { cout<<"Base::fun"<<endl; }};
class Derived : public Base{
public:
void fun(){
-----
cout<<"Derived::fun"<<endl;}};
int main(){
Derived d;
d.fun();
return 0;}
```

已知其执行后的输出结果为：

```
Base::fun
Derived::fun
```

则程序中下划线处应填入的语句是：(B)

A. Base.fun(); B. Base::fun(); C. Base->fun(); D. fun();

6. 有如下程序：

```
#include<iostream>
using namespace std;
class Base {
    int x;
public:
    Base(int n=0): x(n){cout<<n;}
    int getX()const{return x;}};
class Derived: public Base{
    int y;
public:
    Derived(int m, int n): y(m), Base(n){cout<<m;}
    Derived(int m): y(m){cout<<m;}};
int main(){
    Derived d1(3), d2(5,7);
    return 0;}
```

运行时的输出结果是：(C)

A. 375 B. 357 C. 0375 D. 0357

7. 有如下类定义：

```
class XX {
    int xx;
public:
    XX() : xx(0) {cout<<'A'; }
    XX(int n) : xx(n){cout<<'B'; } };
class YY : public XX {
    int yy;
public:
    YY() : yy(0) {cout<<yy;}
    YY(int n): XX(n+1), yy(n) {cout<<yy; }
    YY(int m, int n) : XX(m), yy(n) {cout<<yy; } };
```


下列选项中，输出结果为 A0 的语句是：(D)

A. YY y1(0,0); B. YY y2(1); C. YY y3(0); D. YY y4;

8. 应在下列程序划线处填入的正确语句是 (C)。

```
class Base
{ public:
    void fun(){cout<<"Base::fun"<<endl;}    };
class Derived:public Base
{ void fun()
{    ——//显示调用基类的函数fun()
    cout<<"Derived::fun"<<endl; }    };
```

A. fun(); B. Base.fun(); C. Base::fun(); D. Base->fun();

9. 关于多继承二义性的描述，(D) 是错误的。

- A. 派生类的多个基类中存在同名成员时，派生类对这个成员访问可能出现二义性
- B. 如果一个派生类是从具有两个同名间接基类的两个直接基类派生来的，则派生类对该公共基类的访问可能出现二义性
- C. 解决二义性最常用的方法是使用作用域运算符对成员进行限定
- D. 派生类和它的基类中出现同名函数时，将出现二义性

10. 有如下程序：

```
#include<iostream>
using namespace std;
class AA {
    int k;
protected:
    int n;
    void setK(int k){this->k=k;}
public:
    void setN(int n){this->n=n;} };
class BB: public AA { /*类体略*/ };
int main() {
    BB x;
    x.n=1; //1
    x.setN(2); //2
    x.k=3; //3
    x.setK(4); //4
    return 0; }
```

在标注号码的四条语句中正确的是：(B)

- A. 1 B. 2 C. 3 D. 4

11. 下列对派生类的描述中，(D) 是错的

- A. 一个派生类可以作另一派生类的基类
B. 派生类至少有一个基类
C. 派生类的成员除了它自己的成员外，还包含了它的基类的成员
D. 派生类中继承的基类成员的访问权限到派生类中保持不变

12. 有如下类声明：

```
class MyBASE{
    int k;
public:
    void set(int n){ k=n;}
    int get( )const{ return k; };
class MyDERIVED: protected MyBASE{
protected:
    int j;
public:
    void set(int m, int n){ MyBASE::set(m); j=n;}
    int get( )const{ return MyBASE::get( )+j; };
}
```

则类 MyDERIVED 中保护的数据成员和成员函数的个数是：(B)

- A. 4 B. 3 C. 2 D. 1

13. 使用派生类的主要原因有：(A)

- A. 提高代码的可重用性 B. 提高程序的运行效率
C. 加强类的封装性 D. 实现数据的隐藏

14. 若有如下类定义：

```
class B{
    void fun1(){ }
protected:
    double var1;
public:
    void fun2(){ };
class D:public B{
protected:
    void fun3(){ };
}
```

已知 obj 是类 D 的对象，下列句中不违反类成员访问控制权限的是：(C)

A. obj.fun1(); B. obj.var1; C. obj.fun2(); D. obj.fun3();

15. 在一个派生类对象结束其生命周期时：(A)

- A. 先调用派生类的析构函数后调用基类的析构函数
- B. 先调用基类的析构函数后调用派生类的析构函数
- C. 如果基类没有定义析构函数，则只调用派生类的析构函数
- D. 如果派生类没有定义析构函数，则只调用基类的析构函数

16. 有如下类声明：

```
class XA
{ private: int x;
  public:  XA(int n){ x=n;} };
class XB: public XA
{ private: int y;
  public:  XB(int a,int b); };
```

在构造函数 XB 的下列定义中，正确的是：(B)

- A. XB::XB(int a,int b): x(a),y(b) B. XB::XB(int a,int b): XA(a),y(b)
- C. XB::XB(int a,int b): x(a),XB(b) D. XB::XB(int a,int b): XA(a),XB(b)

17. 假设已经定义好一个类 student，现在要定义类 derived，它是从 student 私有派生的，定义类 derived 的正确写法是：(C)

- A. class derived::student private...;
- B. class derived::student public...;
- C. class derived::private student ...;
- D. class derived:: public student ...;

18. 下列关于虚基类的叙述中，错误的是：(B)

- A. 使用虚基类可以消除由多继承产生的二义性
- B. 声明“class B:virtual public A”说明类 B 为虚基类
- C. 建立派生类对象时，首先调用虚基类的构造函数
- D. 构造派生类对象时，虚基类的构造函数只被调用一次

19. 如果派生类以 protected 方式继承基类，则原基类的 protected 成员和 public 成员在派生类中的访问属性分别是：(D)

- A. public public B. public protected
- C. protected public D. protected protected

20. 有如下程序:

```
#include<iostream>
using namespace std;
class A {
public:
    A( ) { cout << "A"; }
};
class B { public: B( ) { cout << "B"; } };
class C : public A {
    B b;
public:
    C( ) { cout << "C"; }
};
int main( ) { C obj; return 0; }
```

执行后的输出结果是: (D)

- A. CBA B. BAC C. ACB D. ABC

21. 有如下类声明, 则类 MyDERIVED 中保护的数据成员和成员函数的个数是: (B)。

```
class MyBASE
{ private:    int k;
public:      void set(int n){ k=n;}
            int get( )const{ return k;}
};
class MyDERIVED: protected MyBASE
{ protected: int j;
public:      void set(int m, int n){ MyBASE::set(m); j=n;}
            int get( ) const { return MyBASE::get( )+j; }
};
```

- A. 4 B. 3 C. 2 D. 1

22. 建立派生类对象时,3 种构造函数分别是 a(基类的构造函数)、b(成员对象的构造函数)、c(派生类的构造函数) 这 3 种构造函数的调用顺序为: (A)

- A. abc B. acb C. cab D. cba

23. 有如下程序

```
#include <iostream>
using namespace std;
class Base {
protected:
```

```

Base( ){ cout<<'A'; }
Base(char c){ cout<<c; };
class Derived: public Base{
public:
    Derived( char c ){ cout<<c; };
int main( ){
    Derived d1('B');
    return 0;}

```

执行这个程序屏幕上将显示输出: (C)

- A. B B. BA C. AB D. BB

24. 下面描述中, 表达错误的是 (B)

- A. 公有继承时基类中的 public 成员在派生类中仍是 public 的
- B. 公有继承时基类中的 private 成员在派生类中仍是 private 的
- C. 公有继承时基类中的 protected 成员在派生类中仍是 protected 的
- D. 私有继承时基类中的 public 成员在派生类中是 private 的

25. 有如下类声明:

```

class XA{
    int x;
public:
    XA(int n){ x=n;}
};
class XB: public XA{
    int y;
public:
    XB(int a,int b);
};

```

在构造函数 XB 的下列定义中, 正确的是: (B)

- A. XB::XB(int a,int b): x(a), y(b) B. XB::XB(int a,int b): XA(a), y(b)
 C. XB::XB(int a,int b): x(a), XB(b) D. XB::XB(int a,int b): XA(a), XB(b)

26. C++ 中, 下列关于继承的描述, (A) 是错误的。

- A. 继承是基于对象操作的层面而不是类设计的层面上的
- B. 子类可以继承父类的公共行为
- C. 继承是通过重用现有的类来构建新的类的一个过程
- D. 将相关的类组织起来, 从而可以共享类中的共同的数据和操作

27. 可以用 pt.a() 的形式访问派生类对象 pt 的基类成员函数 a，其中 a 是：(D)

- A. 私有继承的公有成员
- B. 公有继承的保护成员
- C. 公有继承的私有成员
- D. 公有继承的公有成员

28. 有如下程序：

```
#include<iostream>
using namespace std;
class Base{
public:
    Base(int x=0){cout<<x;};
class Derived:public Base{
public:
    Derived(int x=0){cout<<x;};
private:
    Base val;};
```

程序的输出结果是：(D)

- A. 0
- B. 1
- C. 01
- D. 001

29. 在 C++ 中，(B) 一定不能创建类的对象和实例。

- A. 虚基类
- B. 抽象类
- C. 基类
- D. 派生类

30. 设置虚基类的目的是：(B)

- A. 简化程序
- B. 消除二义性
- C. 提高运行效率
- D. 减少目标代码

31. 在多继承构造函数定义中，几个基类构造函数用 (C) 分隔

- A. :
- B. ;
- C. ,
- D. ::

```
32. #include<iostream.h>
class A
{ public: A(){
    ~A(){cout<<"A destroy";} };
class B:public A
{ public: B():A(){
    ~B(){cout<<"B destroy";} };
void main(){B obj;}
```

上面的 C++ 程序运行的结果是：(A)

- A. B destroy A destroy
- B. A destroy B destroy
- C. A destroy
- D. B destroy

33. 继承具有 (B)，即当基类本身也是某一个类派生类时，底层的派生类也会自动继承间接基类的成员。

A. 规律性

B. 传递性

C. 重复性

D. 多样性

五、多态性

1. 下面关于虚函数和函数重载的叙述不正确的是：(A)
 - A. 虚函数不是类的成员函数
 - B. 函数重载的调用根据参数的个数、序列来确定，而虚函数依据对象确定
 - C. 函数重载允许非成员函数，而虚函数则不行
 - D. 虚函数实现了 C++ 的多态性
2. 关于运算符重载的下列描述中正确的是：(B)
 - A. 改变优先级
 - B. 保持原有的结合性
 - C. 改变操作数的个数
 - D. 改变语法结构
3. 派生类中虚函数原型的 (D)
 - A. 函数类型可以与基类中虚函数的原型不同
 - B. 参数个数可以与基类中虚函数的原型不同
 - C. 参数类型可以与基类中虚函数的原型不同
 - D. 其他都不对
4. 下面四个选项中，(A) 是用来声明虚函数的。
 - A. virtual
 - B. public
 - C. include
 - D. using namespace
5. 关于下列虚函数的描述中，(C) 是正确的。
 - A. 虚函数是一个 static 存储类的成员函数
 - B. 虚函数是一个非成员函数
 - C. 基类中说明了虚函数后，派生类中可不必将对应的函数说明为虚函数
 - D. 派生类的虚函数与基类的虚函数应具有不同的类型或个数
6. 程序编译时的多态性通过使用 (C) 获得。
 - A. 继承
 - B. 虚函数
 - C. 重载函数
 - D. 析构函数
7. 关于纯虚函数和抽象类的描述中，(C) 是错误的。
 - A. 纯虚数是一种特殊的虚函数，它没有具体实现
 - B. 抽象类中一定具有一个或多个纯虚函数
 - C. 抽象类的派生类中一定不会再有纯虚函数
 - D. 抽象类一般作基类使用，纯虚函数的实现由其派生类给出
8. 关于动态联编的下列描述中，(D) 是错误的
 - A. 动态联编是以虚函数为基础的
 - B. 动态联编是在运行时确定所调用的函数代码的
 - C. 动态联编调用函数操作是指向对象的指针或对象引用
 - D. 动态联编是在编译时确定操作函数的

9. 下面关于虚函数的描述，错误的是：(D)
- A. 在成员函数声明的前面加上 `virtual` 修饰，就可把该函数声明为虚函数
 - B. 基类中说明了虚函数后，派生类中符合重新定义的虚函数要求的同名函数都自动成为虚函数
 - C. 虚函数必须是其所在类的成员函数，不能是友元函数，也不能是静态成员函数
 - D. 基类中说明的纯虚函数在其任何派生类中都必须实现
10. 关于重载运算符的说法中，错误的是：(D)
- A. C++ 只能对已有的运算符进行重载
 - B. 重载不能改变运算符运算对象的个数
 - C. 重载不能改变运算符的优先级，也不能改变运算符的结合性
 - D. 重载运算符的函数可以有默认的参数
11. 在 C++ 中，下列关于虚函数的描述，(A) 是正确的。
- A. 虚函数实现了运行时的多态
 - B. 虚函数只能在基类中实现，而不能在派生类中实现
 - C. 虚函数不需要在基类中声明
 - D. 虚函数只能在派生类中声明
12. 如果在基类中将 `show` 声明为不带返回值的纯虚函数，正确的写法是：(C)
- A. `virtual show()=0;`
 - B. `virtual void show();`
 - C. `virtual void show()=0;`
 - D. `void show()=0 virtual;`
13. 下列关于纯虚函数与抽象类的描述中，错误的是：(C)
- A. 纯虚函数是一种特殊的函数，它允许没有具体的实现。
 - B. 抽象类是指具有纯虚函数的类
 - C. 一个基类的说明中有纯虚函数，该基类的派生类一定不再是抽象类
 - D. 抽象类只能作为基类来使用，其纯虚函数的实现由派生类给出
14. 有如下程序；

```
class Base{
public:
    void output(){cout<<1;}
    virtual void Print(){cout<<'B';}};
class Derived:public Base{
public:
    void output(){cout<<2;}
```

```

    void Print(){cout<<'D';}};
void main(){
    Base *ptr;
    Derived obj;
    ptr=&obj;
    ptr->output();
    ptr->Print();
    return;}

```

程序的输出结果是：(A)

- A. 1D B. 2D C. 1B D. 2B

15. 在 C++ 中，运算符重载不能通过下列 (C) 方式获得。

- A. 重载友元函数 B. 重载成员函数
C. 创造一种新的运算符 D. 重载全局函数

16. 如果一个类中包含一个或多个纯虚函数，则这个类是：(D)

- A. 具体类 B. 抽象类的派生类 C. 虚基类 D. 抽象类

17. 下面关于运算符重载的描述错误的是：(D)

- A. 运算符重载不能改变操作数的个数 B. 运算符重载不能改变运算符的结合性
C. 运算符重载不能改变运算符的优先级 D. 运算符重载可以改变运算符的操作数类型

18. 在 C++ 中，关键字 virtual 主要用于说明 (B)。

- A. 类 B. 虚函数 C. 对象 D. 数据成员

19. 下列关于虚函数的描述中，(D) 是错误的。

- A. 虚函数必须是类的成员函数 B. 虚函数可以有返回值
C. 虚函数在基类中声明 D. 虚函数可以在全局函数中声明

20. 下列关于纯虚函数的叙述，正确的是：(D)

- A. 纯虚函数没有函数体 B. 纯虚函数所在的类是抽象类
C. 纯虚函数不能被调用 D. 以上都正确

21. 若 MyInt 类中只定义了一个 int 类型的成员变量 data，下列 (A) 函数不可能是 MyInt 类的重载运算符函数。

- A. MyInt operator+(int);
B. MyInt operator++(int);
C. MyInt operator--();
D. friend MyInt operator+(MyInt, int);

22. 下列运算符中，在 C++ 中不能被重载的是：(A)

- A. . B. + C. - D. *

23. 下列关于虚函数的叙述中，错误的是：(C)

- A. 虚函数在派生类中必须重新定义 B. 虚函数允许动态联编
C. 虚函数必须是类的成员函数 D. 虚函数可以被继承

24. 下列关于运算符重载的描述中，(D) 是正确的。

- A. 运算符重载可以改变运算符的优先级
B. 运算符重载可以改变运算符的结合性
C. 运算符重载可以改变运算符所需要的操作数个数
D. 运算符重载可以针对用户自定义的类型定义新的运算符

25. 下面 (A) 运算符不能被重载。

- A. . B. + C. - D. *

26. 有如下类定义：

```
class Point {
public:
    Point(int x = 0, int y = 0): x_(x), y_(y) { }
    int getX() const { return x_; }
    int getY() const { return y_; }
    Point operator+(const Point &p) const;
private:
    int x_, y_;
};

Point Point::operator+(const Point &p) const {
    return Point(x_ + p.x_, y_ + p.y_);
}

int main() {
    Point p1(1, 2), p2(3, 4);
    Point p3 = p1 + p2;
    cout << p3.getX() << ", " << p3.getY() << endl;
    return 0;
}
```

程序运行后的输出结果是：(B)

- A. 1, 2 B. 4, 6 C. 3, 4 D. 0, 0

27. 关于虚函数，下面说法正确的是 (D)

- A. 虚函数在基类中必须定义，在派生类中可以不定义
- B. 虚函数在基类中定义后，在派生类中可以重新定义
- C. 虚函数是为了实现静态多态性
- D. 构造函数不能定义为虚函数，析构函数可以定义为虚函数

28. 下面关于运算符重载的描述中，(B) 是错误的。

- A. 可以通过成员函数或友元函数的方式实现
- B. 重载后的运算符的操作数至少应有一个是自定义类型
- C. 对已有的运算符进行重载时，不能改变其操作数的个数
- D. 通过对运算符进行重载，可以实现对自定义类型数据的操作

29. 下列关于纯虚函数的描述中，错误的是：(A)

- A. 一个类中说明了纯虚函数，由该类派生出的所有类仍然是抽象类
- B. 纯虚函数是没有函数体的虚函数
- C. 纯虚函数只能在基类中说明
- D. 如果派生类中没有重新定义纯虚函数，则该派生类也是抽象类

30. 有如下定义：

```
class AA{
    int a;
public:
    AA(int i = 0){a = i;}
    AA operator ++(int){ return AA(a++);}
    AA operator ++(){ return AA(++a);}
    void print(){cout << a ;}
};

void main(){
    AA x(5);
    (x++).print();
    (++x).print();
}
```

程序输出结果为：(B)

- A. 56
- B. 57
- C. 66
- D. 67

31. 下列关于运算符重载的描述中，错误的是：(D)

- A. 可以重载已经存在的运算符
- B. 可以重载为成员函数或友元函数
- C. 通过重载可以扩展运算符的功能
- D. 只能通过成员函数重载运算符

32. 下列关于虚函数的叙述中，正确的是 (C)。

- A. 虚函数在派生类中必须重新定义
- B. 只能通过指向对象的指针或引用来调用虚函数
- C. 静态成员函数不能定义为虚函数
- D. 构造函数和析构函数都可以定义为虚函数

33. 在 C++ 中，不能重载的运算符是 (C)。

- A. []
- B. new
- C. .*
- D. ->*

六、模板与异常处理

1. 有如下函数模板：

```
template<typename T, class U>
T cast(U u){return u;}
```

其功能是将 U 类型数据转换为 T 类型数据。已知 i 为 int 型变量，下列对模板函数 cast 的调用中正确的是：(D)

- A. cast(i);
- B. cast<>(i);
- C. cast<char*,int>(i);
- D. cast<double,int>(i);

2. 模板对类型的参数化提供了很好的支持，因此 (B)

- A. 类模板的主要作用是生成抽象类
- B. 类模板实例化时，编译器将根据给出的模板实参生成一个类
- C. 在类模板中的数据成员都具有同样类型
- D. 类模板中的成员函数都没有返回值

3. 下列关于类模板的模板参数的叙述中，错误的是：(D)

- A. 模板参数可以作为数据成员的类型
- B. 模板参数可以作为成员函数的返回类型
- C. 模板参数可以作为成员函数的参数类型
- D. 模板参数不能作为成员函数的局部变量的类型

4. 下列选项中，属于语法错误的是：(C)

- A. 在计算过程中，出现除数为 0 的情况
- B. 内存空间不够，无法实现指定的操作
- C. 括号不配对
- D. 无法打开输入文件

5. 下列关于模板的描述，不正确的是：(C)

- A. 利用模板机制可进一步提高面向对象程序的可重用性
- B. 模板可实现类型参数化，把类型定义为参数
- C. 函数模板就是模板函数，类模板就是模板类
- D. 利用模板机制可进一步提高面向对象程序的可维护性

6. C++ 处理异常的机制是由 (B) 3 部分组成

- A. 编辑、编译和运行
- B. 检查、抛出和捕获
- C. 编辑、编译和捕获
- D. 检查、抛出和运行

7. 有如下函数模板定义：

```
template<typename T>
T func(T x, T y) { return x*x+y*y; }
```

在下列对 func 的调用中，错误的是：(C)

- A. func(3, 5);
- B. func(3.0, 5.5);
- C. func (3, 5.5);
- D. func<int>(3, 5.5);

8. 下列选项中，属于运行错误的是：(A)

- A. 输入数据时数据类型有错
- B. 语句末尾缺少分号
- C. 变量名未定义
- D. 关键字拼写错误

9. 下列对模板作用的叙述中，错误的是：(C)

- A. 提供代码的可重用性
- B. 提高代码的运行效率
- C. 加强类的封装性
- D. 实现多态性

10. 假设声明了以下的函数模板：

```
template<class T>
T max(T x, T y)
{
    return (x>y)?x:y;
}
```

并定义了 int i; char c; 错误的调用语句是：(D)

- A. max(i,i)
- B. max(c,c)
- C. max((int)c,i)
- D. max(i,c)

11. 下列函数模板的定义中，合法的是：(A)

- A. template<typename T> T abs (T x) return (x <0) ?-x: x;
- B. template class <T> T abs (T x) return (x<0) ?-x:x;
- C. template T<class T.> abs (T x) return (x<0) ?-x:x;
- D. template T abs (T x) return (x<0) ?-x:x;

七、C++ 的流类库与输入输出

1. 命名空间应用于：(B)

- A. 在类外定义类的成员函数
- B. 避免各个不同函数、变量等的名称冲突
- C. 提高代码的执行速度
- D. 提高内容使用率

2. 下面关于文件打开方式中，说法不正确的是：(D)

- A. `ios::app` 打开一个输出文件，用于将数据添加到文件尾部。
- B. `ios::nocreate` 打开一个文件，若文件不存在，则打开失败。
- C. `ios::in` 打开一个文件，以便进行输入操作。
- D. `ios::out` 打开一个文件，以便进行输入操作。

3. 下面关于文件的说法错误的是：(B)

- A. 文件，一般指存放在外部介质上的数据的集合。
- B. 文件，一般指存放在内部介质上的数据的集合。
- C. 操作系统以文件为单位对数据进行管理
- D. 要把数据存储在外部介质上，必须先建立一个文件，才能向它输出数据

4. 以二进制方式打开一个文件（默认为文本文件）的文件打开方式为：(D)

- A. `ios::app`
- B. `ios::ate`
- C. `ios::nocreate`
- D. `ios::binary`

5. 要利用 C++ 流进行文件操作，必须在程序中包含的头文件是：(B)

- A. `iostream`
- B. `fstream`
- C. `strstream`
- D. `omanip`

6. 下面关于文件关闭的说法，错误的是：(D)

- A. 所谓关闭，实际上就是将所打开的磁盘文件与流对象“脱钩”，这样，就不能通过文件流对象对该文件进行输入或输出操作了。
- B. 在进行文件操作后，应该关闭文件，否则可能丢失数据。
- C. 关闭文件可使用 `close` 函数，它不带参数。
- D. `out.close()`；就是将与流对象所关联的磁盘文件关闭。关闭后，文件流对象 `out` 就不能与其他文件进行关联了，也不能对新的文件进行输入或输出。

7. 下列说法中，错误的是：(D)

- A. 在实际应用中，常用磁盘作为数据存放的中介。
- B. 应用程序的输入模块通过键盘或其他输入设备将数据读入磁盘
- C. 从操作系统的角度来说，每一个与主机相连的输入输出设备都可以看作一个文件。
- D. 键盘是输入文件，显示器和打印机是输出文件，手机屏幕只能是输出文件。

8. 如果在函数中定义的局部变量与命名空间中的变量同名时，(B)被隐藏。
- A. 函数中的变量 B. 命名空间中的变量
- C. 两个变量都 D. 两个变量都不
9. 如果程序中使用了 using 命令同时引用了多个命名空间，并且命名空间中存在相同的函数，将出现：(A)
- A. 编译错误 B. 语法错误
- C. 逻辑错误 D. 无法判定错误类型
10. 下列选项中用于输入输出格式控制的头文件为：(D)
- A. iostream B. fstream C. strstream D. iomanip
11. 打开一个输出文件，用于将数据添加到文件尾部的文件打开方式为：(A)
- A. ios::app B. ios::out C. ios::trunc D. ios::binary
12. 下列选项中，(D)是缓冲型的标准出错流对象。
- A. cin B. cout C. cerr D. clog
13. 要说明标识符是属于哪个命名空间时，需要在标识符和命名空间名字之间加上：(A)
- A. :: B. -> C. . D. ()
14. 对于语句 `cout<<x<<endl;`, 错误的是描述是：(D)
- A. “cout”是一个输出流对象 B. “endl”的作用是输出回车换行
- C. “x”是一个变量 D. “<<”称作提取运算符

八、STL 标准模板库

1. 标准模板库的英文缩写为: (B)

- A. SDT B. STL C. SLL D. STD

2. 下列选项中, 哪一项是面向对象版本的指针? (C)

- A. 容器 B. for_each C. 迭代器 D. sort 函数

3. 下列选项中, 哪一项不是标准模板库的三个基本组成部分? (D)

- A. 容器 B. 迭代器 C. 算法 D. 循环

4. 下列选项中, 哪一项容器中每个元素的值唯一, 且系统能根据元素的值自动进行排序。(C)

- A. list B. stack C. set D. string

5. 下面程序的运行结果为:

```
#include <iostream>
#include<vector>
#include<algorithm>
using namespace std;

void PrintLine(double &d)
{ cout<<d<<endl;}

int main()
{
    vector<double> s(2);
    s[0]=2.2;
    s[1]=1.23;
    s.insert(s.begin(),3.3);
    for_each(s.begin(),s.end(),PrintLine);
    return 0;
}
```

(A)

- A. 3.3 2.2 1.23 B. 2.2 1.23 3.3 C. 2.2 3.3 1.23 D. 3.3 1.23 2.2

6. 下面程序的运行结果为: (A)

```
#include <iostream>
#include<vector>
using namespace std;

int main()
```

```
{  
    vector<double> s(2);  
    s[1]=1.23;  
    s.insert(s.begin(),3.3);  
    cout<<s.size()<<endl;  
    s.push_back(6.7);  
    cout<<s[2]<<endl;  
    cout<<s.size()<<endl;  
    return 0;  
}
```

A. 3 1.23 4

B. 3 3.3 4

C. 2 3.3 3

D. 2 1.23 3

7. 下列选项中，哪一项是后进先出的容器？(B)

A. list

B. stack

C. map

D. set

九、面向对象程序设计方法与实例

```
1.  #include<iostream.h>
    class A{
        int x;
        public:
        A(int x1){x=x1;}
        void print( ){cout<<"x="<<x;} };
    class B: private A{
        int y;
        public:
        B(int x1,int y1):A(x1){y=y1;}
        ①;
    };
    int main( )
    {
        B b(10,20);
        b.print( );
        return 0;
    }
```

要想程序能够正确执行，①处的代码是：（C）

- | | |
|---------------------|----------------|
| A. void A::print; | B. A::print(); |
| C. void A::print(); | D. A::print; |

2025 年 5 月 16 日